autoencoder

March 3, 2026

[1]:
```
import keras
from keras import layers

# This is the size of our encoded representations
encoding_dim = 32  # 32 floats -> compression of factor 24.5, assuming the␣
↪input is 784 floats

# This is our input image
input_img = keras.Input(shape=(784,))
# "encoded" is the encoded representation of the input
encoded = layers.Dense(encoding_dim, activation='relu')(input_img)

# "decoded" is the lossy reconstruction of the input
decoded = layers.Dense(784, activation='sigmoid')(encoded)
# This model maps an input to its reconstruction
autoencoder = keras.Model(input_img, decoded)
```

```
2026-03-03 11:20:51.517788: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:477] Unable to register
cuFFT factory: Attempting to register factory for plugin cuFFT when one has
already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
E0000 00:00:1772533251.531241   79924 cuda_dnn.cc:8310] Unable to register cuDNN
factory: Attempting to register factory for plugin cuDNN when one has already
been registered
E0000 00:00:1772533251.535731   79924 cuda_blas.cc:1418] Unable to register
cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has
already been registered
2026-03-03 11:20:51.552383: I tensorflow/core/platform/cpu_feature_guard.cc:210]
This TensorFlow binary is optimized to use available CPU instructions in
performance-critical operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild
TensorFlow with the appropriate compiler flags.
I0000 00:00:1772533253.731810   79924 gpu_device.cc:2022] Created device
/job:localhost/replica:0/task:0/device:GPU:0 with 1033 MB memory:  -> device: 0,
name: NVIDIA GeForce RTX 3090, pci bus id: 0000:01:00.0, compute capability: 8.6
```

[2]:
```
# This model maps an input to its encoded representation
encoder = keras.Model(input_img, encoded)
```

[3]:
```
# This is our encoded (32-dimensional) input
encoded_input = keras.Input(shape=(encoding_dim,))
# Retrieve the last layer of the autoencoder model
decoder_layer = autoencoder.layers[-1]
# Create the decoder model
decoder = keras.Model(encoded_input, decoder_layer(encoded_input))
```

We'll configure our model to use a per-pixel binary crossentropy loss, and the Adam optimizer:

[4]:
```
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')
```

[5]:
```
from keras.datasets import mnist
import numpy as np
(x_train, _), (x_test, _) = mnist.load_data()
```

We will normalize all values between 0 and 1 and we will flatten the 28x28 images into vectors of size 784.

[6]:
```
x_train = x_train.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.
x_train = x_train.reshape((len(x_train), np.prod(x_train.shape[1:])))
x_test = x_test.reshape((len(x_test), np.prod(x_test.shape[1:])))
print(x_train.shape)
print(x_test.shape)
```

```
(60000, 784)
(10000, 784)
```

[7]:
```
autoencoder.fit(x_train, x_train, # se pone dos veces xtrain es porque el␣
 ↪primero sirve para hacere encoder i leugo decorder, i el seguno es como␣
 ↪"etiqueta" de referencia para comparar.
                epochs=50,
                batch_size=256,
                shuffle=True,
                validation_data=(x_test, x_test))
```

```
Epoch 1/50

WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
I0000 00:00:1772533255.418453   80012 service.cc:148] XLA service 0x7054e0004fb0
initialized for platform CUDA (this does not guarantee that XLA will be used).
Devices:
I0000 00:00:1772533255.418492   80012 service.cc:156]   StreamExecutor device
(0): NVIDIA GeForce RTX 3090, Compute Capability 8.6
2026-03-03 11:20:55.431667: I
tensorflow/compiler/mlir/tensorflow/utils/dump_mlir_util.cc:268] disabling MLIR
```

```
crash reproducer, set env var `MLIR_CRASH_REPRODUCER_DIRECTORY` to enable.
I0000 00:00:1772533255.500207   80012 cuda_dnn.cc:529] Loaded cuDNN version
90800

 37/235            0s 4ms/step -
loss: 0.6157

I0000 00:00:1772533256.953439   80012 device_compiler.h:188] Compiled cluster
using XLA!  This line is logged at most once for the lifetime of the process.

235/235            6s 16ms/step -
loss: 0.3849 - val_loss: 0.1881
Epoch 2/50
235/235            6s 6ms/step -
loss: 0.1791 - val_loss: 0.1540
Epoch 3/50
235/235            1s 5ms/step -
loss: 0.1501 - val_loss: 0.1344
Epoch 4/50
235/235            1s 6ms/step -
loss: 0.1320 - val_loss: 0.1215
Epoch 5/50
235/235            2s 6ms/step -
loss: 0.1206 - val_loss: 0.1125
Epoch 6/50
235/235            1s 5ms/step -
loss: 0.1124 - val_loss: 0.1066
Epoch 7/50
235/235            1s 5ms/step -
loss: 0.1065 - val_loss: 0.1021
Epoch 8/50
235/235            1s 6ms/step -
loss: 0.1026 - val_loss: 0.0991
Epoch 9/50
235/235            1s 6ms/step -
loss: 0.0996 - val_loss: 0.0970
Epoch 10/50
235/235            1s 6ms/step -
loss: 0.0979 - val_loss: 0.0955
Epoch 11/50
235/235            2s 5ms/step -
loss: 0.0966 - val_loss: 0.0948
Epoch 12/50
235/235            1s 5ms/step -
loss: 0.0957 - val_loss: 0.0939
Epoch 13/50
235/235            1s 5ms/step -
loss: 0.0949 - val_loss: 0.0935
Epoch 14/50
```

```
235/235            2s 7ms/step -
loss: 0.0946 - val_loss: 0.0931
Epoch 15/50
235/235            1s 5ms/step -
loss: 0.0942 - val_loss: 0.0929
Epoch 16/50
235/235            1s 5ms/step -
loss: 0.0940 - val_loss: 0.0928
Epoch 17/50
235/235            1s 6ms/step -
loss: 0.0937 - val_loss: 0.0926
Epoch 18/50
235/235            3s 6ms/step -
loss: 0.0938 - val_loss: 0.0924
Epoch 19/50
235/235            1s 5ms/step -
loss: 0.0938 - val_loss: 0.0924
Epoch 20/50
235/235            1s 5ms/step -
loss: 0.0936 - val_loss: 0.0923
Epoch 21/50
235/235            1s 6ms/step -
loss: 0.0934 - val_loss: 0.0922
Epoch 22/50
235/235            1s 6ms/step -
loss: 0.0933 - val_loss: 0.0921
Epoch 23/50
235/235            3s 6ms/step -
loss: 0.0934 - val_loss: 0.0921
Epoch 24/50
235/235            1s 6ms/step -
loss: 0.0934 - val_loss: 0.0920
Epoch 25/50
235/235            1s 6ms/step -
loss: 0.0933 - val_loss: 0.0920
Epoch 26/50
235/235            1s 6ms/step -
loss: 0.0930 - val_loss: 0.0920
Epoch 27/50
235/235            1s 5ms/step -
loss: 0.0929 - val_loss: 0.0920
Epoch 28/50
235/235            3s 6ms/step -
loss: 0.0934 - val_loss: 0.0919
Epoch 29/50
235/235            3s 6ms/step -
loss: 0.0930 - val_loss: 0.0919
Epoch 30/50
```

```
235/235            2s 5ms/step -
loss: 0.0932 - val_loss: 0.0919
Epoch 31/50
235/235            1s 5ms/step -
loss: 0.0932 - val_loss: 0.0919
Epoch 32/50
235/235            1s 6ms/step -
loss: 0.0931 - val_loss: 0.0919
Epoch 33/50
235/235            2s 6ms/step -
loss: 0.0928 - val_loss: 0.0918
Epoch 34/50
235/235            1s 5ms/step -
loss: 0.0931 - val_loss: 0.0919
Epoch 35/50
235/235            1s 5ms/step -
loss: 0.0929 - val_loss: 0.0917
Epoch 36/50
235/235            1s 5ms/step -
loss: 0.0929 - val_loss: 0.0918
Epoch 37/50
235/235            1s 6ms/step -
loss: 0.0929 - val_loss: 0.0918
Epoch 38/50
235/235            1s 5ms/step -
loss: 0.0929 - val_loss: 0.0918
Epoch 39/50
235/235            1s 6ms/step -
loss: 0.0927 - val_loss: 0.0918
Epoch 40/50
235/235            2s 5ms/step -
loss: 0.0928 - val_loss: 0.0917
Epoch 41/50
235/235            1s 5ms/step -
loss: 0.0928 - val_loss: 0.0917
Epoch 42/50
235/235            1s 5ms/step -
loss: 0.0927 - val_loss: 0.0917
Epoch 43/50
235/235            1s 6ms/step -
loss: 0.0928 - val_loss: 0.0917
Epoch 44/50
235/235            1s 5ms/step -
loss: 0.0928 - val_loss: 0.0917
Epoch 45/50
235/235            1s 5ms/step -
loss: 0.0928 - val_loss: 0.0917
Epoch 46/50
```

```
235/235              1s 5ms/step -
loss: 0.0929 - val_loss: 0.0917
Epoch 47/50
235/235              1s 6ms/step -
loss: 0.0927 - val_loss: 0.0917
Epoch 48/50
235/235              1s 5ms/step -
loss: 0.0925 - val_loss: 0.0916
Epoch 49/50
235/235              1s 5ms/step -
loss: 0.0928 - val_loss: 0.0917
Epoch 50/50
235/235              1s 5ms/step -
loss: 0.0929 - val_loss: 0.0916
```

[7]:
```
<keras.src.callbacks.history.History at 0x70570eb193a0>
```

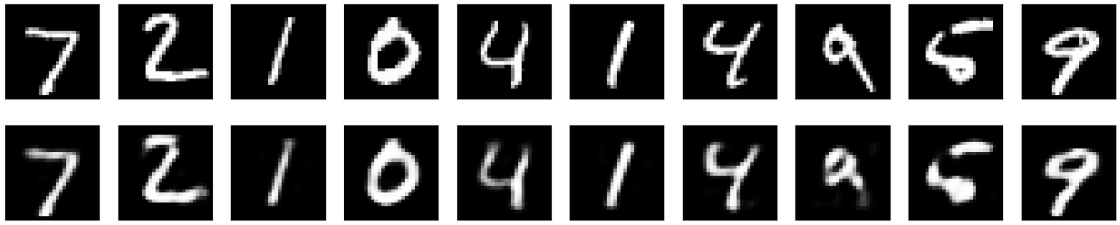
We can try to visualize the reconstructed inputs and the encoded representations. We will use Matplotlib.

[8]:
```
# Encode and decode some digits
# Note that we take them from the *test* set
encoded_imgs = encoder.predict(x_test)
decoded_imgs = decoder.predict(encoded_imgs)
auto= autoencoder.predict(x_test)
```

```
313/313              1s 2ms/step
313/313              1s 2ms/step
313/313              1s 3ms/step
```

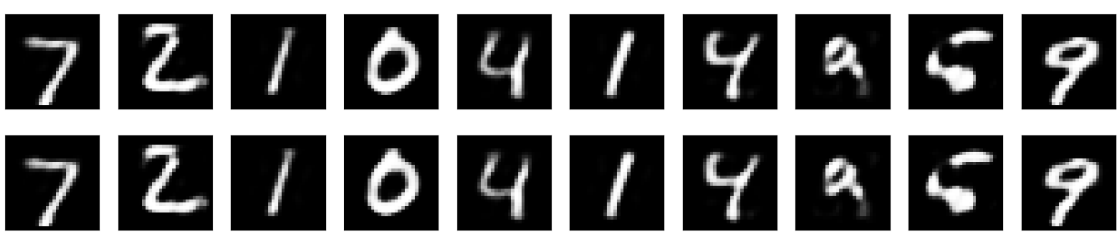
[9]:
```
import matplotlib.pyplot as plt
# aqui se esta ploteando el test original i el test de la salida del decoder
n = 10  # How many digits we will display
plt.figure(figsize=(20, 4))
for i in range(n):
    # Display original
    ax = plt.subplot(2, n, i + 1)
    plt.imshow(x_test[i].reshape(28, 28))
    plt.gray()
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)

    # Display reconstruction
    ax = plt.subplot(2, n, i + 1 + n)
    plt.imshow(decoded_imgs[i].reshape(28, 28))
    plt.gray()
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
plt.show()
```



[10]:
```
# aqui se esta ploteando la salida decoder i el test de la salida del␣
↪autoencoder
#observamos que es lo mismo
n = 10  # How many digits we will display
plt.figure(figsize=(20, 4))
for i in range(n):
    # Display original
    ax = plt.subplot(2, n, i + 1)
    plt.imshow(decoded_imgs[i].reshape(28, 28))
    plt.gray()
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)

    # Display reconstruction
    ax = plt.subplot(2, n, i + 1 + n)
    plt.imshow(auto[i].reshape(28, 28))
    plt.gray()
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
plt.show()
```



Here's what we get. The top row is the original digits, and the bottom row is the reconstructed digits. We are losing quite a bit of detail with this basic approach.

Deep autoencoder We do not have to limit ourselves to a single layer as encoder or decoder, we could instead use a stack of layers, such as:

[11]:
```
input_img = keras.Input(shape=(784,))
encoded = layers.Dense(128, activation='relu')(input_img)
encoded = layers.Dense(64, activation='relu')(encoded)
encoded = layers.Dense(32, activation='relu')(encoded)

decoded = layers.Dense(64, activation='relu')(encoded)
decoded = layers.Dense(128, activation='relu')(decoded)
decoded = layers.Dense(784, activation='sigmoid')(decoded)
```

[12]:
```
autoencoder = keras.Model(input_img, decoded)
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')

autoencoder.fit(x_train, x_train,
                epochs=100,
                batch_size=256,
                shuffle=True,
                validation_data=(x_test, x_test))
```

```
Epoch 1/100

2026-03-03 11:22:25.470874: I
external/local_xla/xla/stream_executor/cuda/cuda_asm_compiler.cc:397] ptxas
warning : Registers are spilled to local memory in function
'gemm_fusion_dot_224', 24 bytes spill stores, 24 bytes spill loads

2026-03-03 11:22:25.840648: I
external/local_xla/xla/stream_executor/cuda/cuda_asm_compiler.cc:397] ptxas
warning : Registers are spilled to local memory in function
'gemm_fusion_dot_238', 8 bytes spill stores, 8 bytes spill loads

2026-03-03 11:22:26.210037: I
external/local_xla/xla/stream_executor/cuda/cuda_asm_compiler.cc:397] ptxas
warning : Registers are spilled to local memory in function
'gemm_fusion_dot_245', 332 bytes spill stores, 332 bytes spill loads

2026-03-03 11:22:26.888080: I
external/local_xla/xla/stream_executor/cuda/cuda_asm_compiler.cc:397] ptxas
warning : Registers are spilled to local memory in function
'gemm_fusion_dot_245', 360 bytes spill stores, 360 bytes spill loads

2026-03-03 11:22:27.206454: I
external/local_xla/xla/stream_executor/cuda/cuda_asm_compiler.cc:397] ptxas
warning : Registers are spilled to local memory in function
'gemm_fusion_dot_252', 20 bytes spill stores, 20 bytes spill loads

2026-03-03 11:22:27.874115: I
external/local_xla/xla/stream_executor/cuda/cuda_asm_compiler.cc:397] ptxas
warning : Registers are spilled to local memory in function
```

```
'gemm_fusion_dot_371_0', 88 bytes spill stores, 88 bytes spill loads

226/235            0s 4ms/step -
loss: 0.3494

2026-03-03 11:22:31.678147: I
external/local_xla/xla/stream_executor/cuda/cuda_asm_compiler.cc:397] ptxas
warning : Registers are spilled to local memory in function
'gemm_fusion_dot_224', 80 bytes spill stores, 96 bytes spill loads

2026-03-03 11:22:31.809109: I
external/local_xla/xla/stream_executor/cuda/cuda_asm_compiler.cc:397] ptxas
warning : Registers are spilled to local memory in function
'gemm_fusion_dot_224', 12 bytes spill stores, 12 bytes spill loads

2026-03-03 11:22:32.100664: I
external/local_xla/xla/stream_executor/cuda/cuda_asm_compiler.cc:397] ptxas
warning : Registers are spilled to local memory in function
'gemm_fusion_dot_245', 168 bytes spill stores, 168 bytes spill loads

2026-03-03 11:22:32.888783: I
external/local_xla/xla/stream_executor/cuda/cuda_asm_compiler.cc:397] ptxas
warning : Registers are spilled to local memory in function
'gemm_fusion_dot_252', 112 bytes spill stores, 160 bytes spill loads

235/235            15s 37ms/step -
loss: 0.3452 - val_loss: 0.1659
Epoch 2/100
235/235            8s 6ms/step -
loss: 0.1570 - val_loss: 0.1350
Epoch 3/100
235/235            2s 6ms/step -
loss: 0.1339 - val_loss: 0.1243
Epoch 4/100
235/235            1s 6ms/step -
loss: 0.1236 - val_loss: 0.1163
Epoch 5/100
235/235            1s 6ms/step -
loss: 0.1160 - val_loss: 0.1111
Epoch 6/100
235/235            1s 6ms/step -
loss: 0.1116 - val_loss: 0.1078
Epoch 7/100
235/235            1s 6ms/step -
loss: 0.1089 - val_loss: 0.1055
Epoch 8/100
235/235            1s 6ms/step -
```

```
loss: 0.1062 - val_loss: 0.1031
Epoch 9/100
235/235            1s 6ms/step -
loss: 0.1038 - val_loss: 0.1016
Epoch 10/100
235/235            2s 6ms/step -
loss: 0.1027 - val_loss: 0.1005
Epoch 11/100
235/235            1s 6ms/step -
loss: 0.1009 - val_loss: 0.0991
Epoch 12/100
235/235            3s 6ms/step -
loss: 0.0996 - val_loss: 0.0976
Epoch 13/100
235/235            2s 6ms/step -
loss: 0.0980 - val_loss: 0.0970
Epoch 14/100
235/235            1s 6ms/step -
loss: 0.0972 - val_loss: 0.0954
Epoch 15/100
235/235            2s 6ms/step -
loss: 0.0958 - val_loss: 0.0939
Epoch 16/100
235/235            1s 5ms/step -
loss: 0.0948 - val_loss: 0.0931
Epoch 17/100
235/235            1s 6ms/step -
loss: 0.0937 - val_loss: 0.0930
Epoch 18/100
235/235            1s 5ms/step -
loss: 0.0931 - val_loss: 0.0921
Epoch 19/100
235/235            1s 6ms/step -
loss: 0.0926 - val_loss: 0.0912
Epoch 20/100
235/235            1s 6ms/step -
loss: 0.0919 - val_loss: 0.0905
Epoch 21/100
235/235            3s 6ms/step -
loss: 0.0912 - val_loss: 0.0901
Epoch 22/100
235/235            1s 6ms/step -
loss: 0.0908 - val_loss: 0.0899
Epoch 23/100
235/235            1s 5ms/step -
loss: 0.0903 - val_loss: 0.0899
Epoch 24/100
235/235            1s 6ms/step -
```

```
loss: 0.0899 - val_loss: 0.0890
Epoch 25/100
235/235            1s 6ms/step -
loss: 0.0891 - val_loss: 0.0886
Epoch 26/100
235/235            1s 6ms/step -
loss: 0.0890 - val_loss: 0.0880
Epoch 27/100
235/235            3s 6ms/step -
loss: 0.0886 - val_loss: 0.0877
Epoch 28/100
235/235            1s 6ms/step -
loss: 0.0885 - val_loss: 0.0875
Epoch 29/100
235/235            1s 5ms/step -
loss: 0.0879 - val_loss: 0.0872
Epoch 30/100
235/235            1s 5ms/step -
loss: 0.0878 - val_loss: 0.0869
Epoch 31/100
235/235            1s 6ms/step -
loss: 0.0874 - val_loss: 0.0866
Epoch 32/100
235/235            1s 6ms/step -
loss: 0.0872 - val_loss: 0.0865
Epoch 33/100
235/235            1s 6ms/step -
loss: 0.0869 - val_loss: 0.0863
Epoch 34/100
235/235            1s 5ms/step -
loss: 0.0867 - val_loss: 0.0861
Epoch 35/100
235/235            1s 6ms/step -
loss: 0.0866 - val_loss: 0.0859
Epoch 36/100
235/235            3s 6ms/step -
loss: 0.0860 - val_loss: 0.0858
Epoch 37/100
235/235            3s 6ms/step -
loss: 0.0861 - val_loss: 0.0859
Epoch 38/100
235/235            1s 6ms/step -
loss: 0.0858 - val_loss: 0.0857
Epoch 39/100
235/235            3s 6ms/step -
loss: 0.0859 - val_loss: 0.0850
Epoch 40/100
235/235            1s 6ms/step -
```

```
loss: 0.0855 - val_loss: 0.0852
Epoch 41/100
235/235            1s 5ms/step -
loss: 0.0853 - val_loss: 0.0849
Epoch 42/100
235/235            1s 6ms/step -
loss: 0.0852 - val_loss: 0.0844
Epoch 43/100
235/235            1s 5ms/step -
loss: 0.0849 - val_loss: 0.0844
Epoch 44/100
235/235            3s 5ms/step -
loss: 0.0849 - val_loss: 0.0843
Epoch 45/100
235/235            3s 6ms/step -
loss: 0.0845 - val_loss: 0.0843
Epoch 46/100
235/235            3s 6ms/step -
loss: 0.0844 - val_loss: 0.0839
Epoch 47/100
235/235            1s 6ms/step -
loss: 0.0843 - val_loss: 0.0839
Epoch 48/100
235/235            3s 6ms/step -
loss: 0.0842 - val_loss: 0.0839
Epoch 49/100
235/235            1s 6ms/step -
loss: 0.0840 - val_loss: 0.0839
Epoch 50/100
235/235            3s 6ms/step -
loss: 0.0841 - val_loss: 0.0837
Epoch 51/100
235/235            3s 6ms/step -
loss: 0.0840 - val_loss: 0.0841
Epoch 52/100
235/235            1s 5ms/step -
loss: 0.0840 - val_loss: 0.0833
Epoch 53/100
235/235            1s 6ms/step -
loss: 0.0838 - val_loss: 0.0832
Epoch 54/100
235/235            1s 6ms/step -
loss: 0.0836 - val_loss: 0.0832
Epoch 55/100
235/235            1s 6ms/step -
loss: 0.0835 - val_loss: 0.0829
Epoch 56/100
235/235            1s 6ms/step -
```

```
loss: 0.0832 - val_loss: 0.0831
Epoch 57/100
235/235            1s 6ms/step -
loss: 0.0831 - val_loss: 0.0829
Epoch 58/100
235/235            1s 6ms/step -
loss: 0.0832 - val_loss: 0.0830
Epoch 59/100
235/235            1s 6ms/step -
loss: 0.0831 - val_loss: 0.0828
Epoch 60/100
235/235            1s 6ms/step -
loss: 0.0828 - val_loss: 0.0827
Epoch 61/100
235/235            1s 5ms/step -
loss: 0.0827 - val_loss: 0.0825
Epoch 62/100
235/235            1s 6ms/step -
loss: 0.0829 - val_loss: 0.0825
Epoch 63/100
235/235            1s 6ms/step -
loss: 0.0827 - val_loss: 0.0826
Epoch 64/100
235/235            1s 6ms/step -
loss: 0.0828 - val_loss: 0.0823
Epoch 65/100
235/235            1s 6ms/step -
loss: 0.0825 - val_loss: 0.0822
Epoch 66/100
235/235            1s 6ms/step -
loss: 0.0826 - val_loss: 0.0823
Epoch 67/100
235/235            1s 6ms/step -
loss: 0.0826 - val_loss: 0.0821
Epoch 68/100
235/235            1s 6ms/step -
loss: 0.0825 - val_loss: 0.0821
Epoch 69/100
235/235            3s 6ms/step -
loss: 0.0823 - val_loss: 0.0820
Epoch 70/100
235/235            1s 6ms/step -
loss: 0.0823 - val_loss: 0.0819
Epoch 71/100
235/235            2s 6ms/step -
loss: 0.0824 - val_loss: 0.0818
Epoch 72/100
235/235            1s 6ms/step -
```

```
loss: 0.0821 - val_loss: 0.0818
Epoch 73/100
235/235            3s 6ms/step -
loss: 0.0824 - val_loss: 0.0819
Epoch 74/100
235/235            3s 6ms/step -
loss: 0.0822 - val_loss: 0.0818
Epoch 75/100
235/235            1s 5ms/step -
loss: 0.0820 - val_loss: 0.0817
Epoch 76/100
235/235            1s 6ms/step -
loss: 0.0819 - val_loss: 0.0818
Epoch 77/100
235/235            1s 5ms/step -
loss: 0.0820 - val_loss: 0.0818
Epoch 78/100
235/235            1s 6ms/step -
loss: 0.0819 - val_loss: 0.0816
Epoch 79/100
235/235            1s 5ms/step -
loss: 0.0819 - val_loss: 0.0818
Epoch 80/100
235/235            1s 6ms/step -
loss: 0.0817 - val_loss: 0.0817
Epoch 81/100
235/235            3s 6ms/step -
loss: 0.0819 - val_loss: 0.0818
Epoch 82/100
235/235            1s 6ms/step -
loss: 0.0819 - val_loss: 0.0817
Epoch 83/100
235/235            2s 6ms/step -
loss: 0.0819 - val_loss: 0.0815
Epoch 84/100
235/235            3s 6ms/step -
loss: 0.0817 - val_loss: 0.0814
Epoch 85/100
235/235            1s 6ms/step -
loss: 0.0817 - val_loss: 0.0814
Epoch 86/100
235/235            2s 6ms/step -
loss: 0.0817 - val_loss: 0.0814
Epoch 87/100
235/235            1s 6ms/step -
loss: 0.0816 - val_loss: 0.0816
Epoch 88/100
235/235            1s 6ms/step -
```

```
loss: 0.0817 - val_loss: 0.0814
Epoch 89/100
235/235            1s 6ms/step -
loss: 0.0817 - val_loss: 0.0815
Epoch 90/100
235/235            1s 5ms/step -
loss: 0.0817 - val_loss: 0.0816
Epoch 91/100
235/235            2s 7ms/step -
loss: 0.0815 - val_loss: 0.0813
Epoch 92/100
235/235            1s 5ms/step -
loss: 0.0815 - val_loss: 0.0812
Epoch 93/100
235/235            1s 6ms/step -
loss: 0.0815 - val_loss: 0.0814
Epoch 94/100
235/235            1s 6ms/step -
loss: 0.0814 - val_loss: 0.0812
Epoch 95/100
235/235            1s 6ms/step -
loss: 0.0815 - val_loss: 0.0811
Epoch 96/100
235/235            1s 5ms/step -
loss: 0.0815 - val_loss: 0.0812
Epoch 97/100
235/235            1s 6ms/step -
loss: 0.0815 - val_loss: 0.0811
Epoch 98/100
235/235            1s 6ms/step -
loss: 0.0814 - val_loss: 0.0812
Epoch 99/100
235/235            1s 6ms/step -
loss: 0.0815 - val_loss: 0.0814
Epoch 100/100
235/235            3s 6ms/step -
loss: 0.0812 - val_loss: 0.0813
```

[12]:
```
<keras.src.callbacks.history.History at 0x70565b97efc0>
```

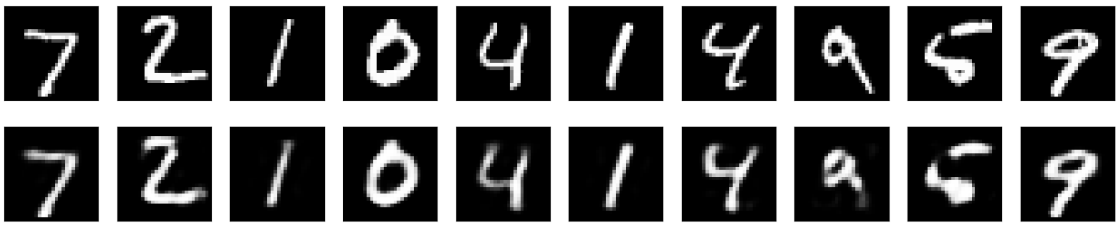
[13]:
```
# Encode and decode some digits
# Note that we take them from the *test* set
encoded_imgs = encoder.predict(x_test)
decoded_imgs = decoder.predict(encoded_imgs)
```

```
313/313            1s 2ms/step
313/313            1s 2ms/step
```

[14]:
```
import matplotlib.pyplot as plt

n = 10  # How many digits we will display
plt.figure(figsize=(20, 4))
for i in range(n):
    # Display original
    ax = plt.subplot(2, n, i + 1)
    plt.imshow(x_test[i].reshape(28, 28))
    plt.gray()
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)

    # Display reconstruction
    ax = plt.subplot(2, n, i + 1 + n)
    plt.imshow(decoded_imgs[i].reshape(28, 28))
    plt.gray()
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
plt.show()
```



[15]:
```
#Hacemos PCA i t-SNE (t-Distributed Stochastic Neighbor Embedding)
# método no lineal de viasualización. Sirve para visualizar grupos(clusters)
# Cargar las etiquetas
(_, y_train), (_, y_test) = mnist.load_data()
```

[16]:
```
data_to_project = encoded_imgs
v_labels = y_test
k = 8000
```

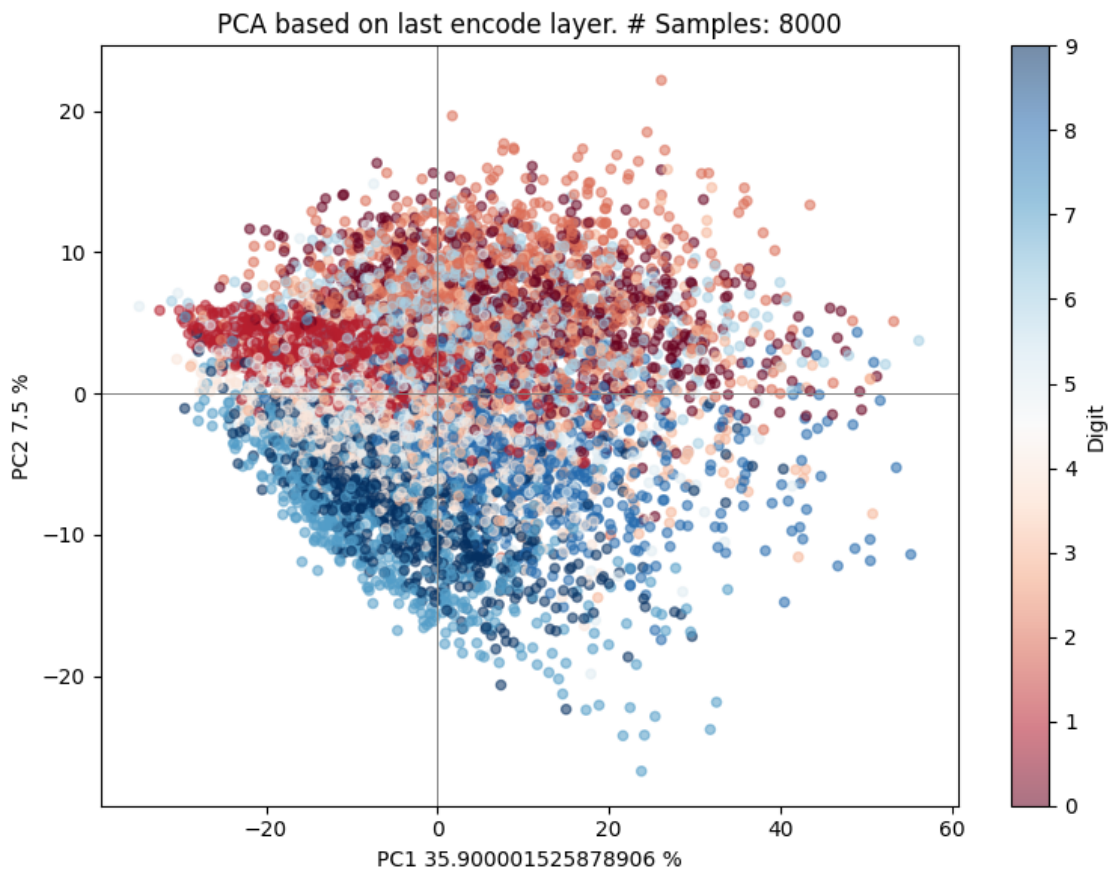
[17]:
```
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
pca = PCA(n_components=2)
z_pca = pca.fit_transform(data_to_project[:k])

# Varianza explicada
loads = np.round(pca.explained_variance_ratio_ * 100, 1)
```

```
plt.figure(figsize=(8, 6))
scatter = plt.scatter(
    z_pca[:, 0],
    z_pca[:, 1],
    c=v_labels[:k],
    cmap="RdBu",
    alpha=0.55,
    s=20
)

plt.axhline(0, color="gray", linewidth=0.7)
plt.axvline(0, color="gray", linewidth=0.7)

plt.xlabel(f"PC1 {loads[0]} %")
plt.ylabel(f"PC2 {loads[1]} %")
plt.title(f"PCA based on last encode layer. # Samples: {k}")
plt.colorbar(scatter, label="Digit")
plt.tight_layout()
plt.show()
```

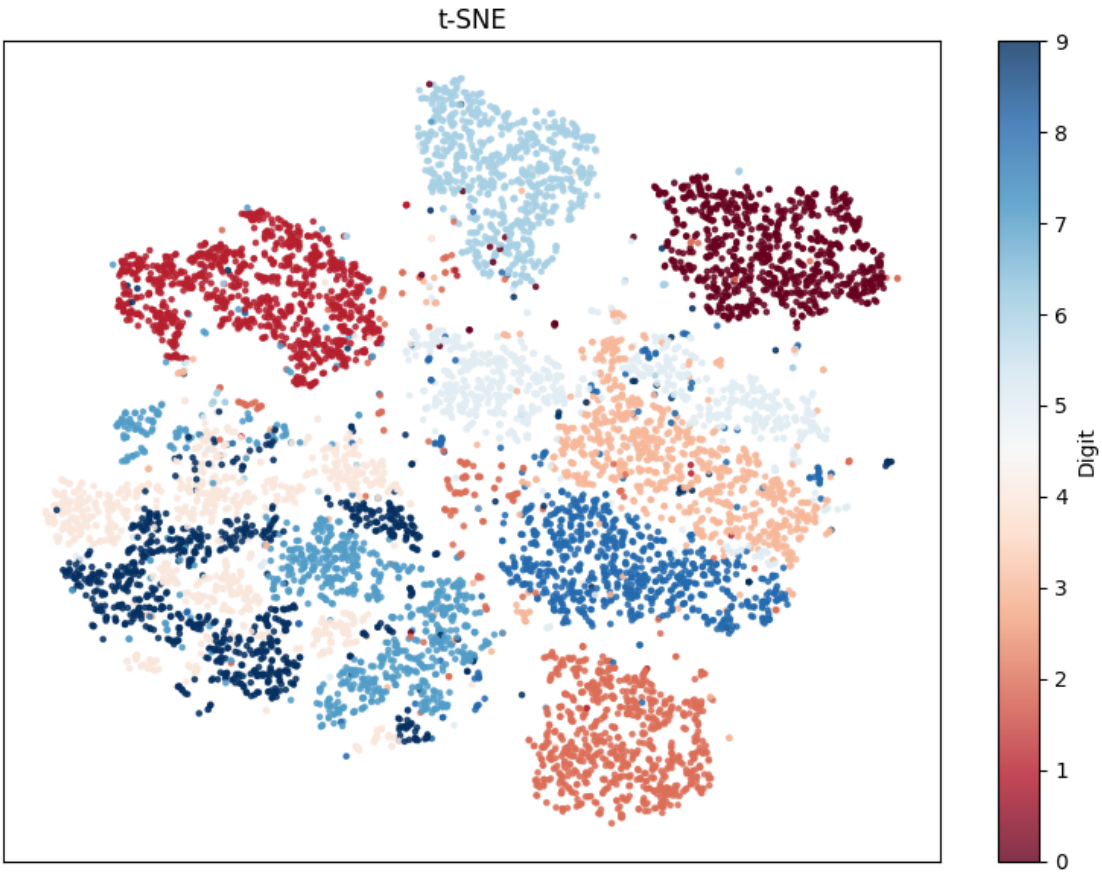


[18]:
```
tsne = TSNE(
    n_components=2,
    perplexity=30,
    learning_rate="auto",
    init="pca",
    random_state=123456
)

z_tsne = tsne.fit_transform(data_to_project[:k])
```

[19]:
```
plt.figure(figsize=(8, 6))
scatter = plt.scatter(
    z_tsne[:, 0],
    z_tsne[:, 1],
    c=v_labels[:k],
    cmap="RdBu",
    s=5,
    alpha=0.8
)

plt.title("t-SNE")
plt.xticks([])
plt.yticks([])
plt.colorbar(scatter, label="Digit")
plt.tight_layout()
plt.show()
```


t-SNE
Digit
0
1
2
3
4
5
6
7
8
9